Lecture 32 — Network Flow I: Max Flow, Min Cut, Bipartite Matching

> Year Curriculum · Prerequisites: Week 16 (BFS/DFS, adjacency lists), Week 17 (shortest paths), Week 19 (cuts in MST context helps) · Estimated time: 3 hours

1. Overview

Network flow is a modeling language: a surprising number of problems — routing data, assigning workers to jobs, choosing which roads to block, finding disjoint paths, scheduling with capacities — become "push as much flow from s to t as possible" once the right graph is drawn. This lecture covers:

- the definitions and the **residual graph**, which makes greedy augmentation correct;
- two algorithms: **Edmonds-Karp** (simple) and **Dinic** (fast, the contest default);
- the **max-flow min-cut theorem** and how to extract a minimum cut;
- standard modeling tricks;
- **bipartite matching**, via flow or via Kuhn's algorithm, and **König's theorem**.

Min-cost flow, Hopcroft-Karp, lower bounds and harder min-cut models are in Nationals lecture 5.

2. Definitions

A **flow network** is a directed graph $G = (V, E)$ with a source s, a sink t, and a capacity $c(u, v) \geq 0$ on each edge. A **flow** is a function f on edges with

- **capacity constraint**: $0 \leq f(u, v) \leq c(u, v)$;
- **conservation**: for every vertex v other than s and t, the total flow into v equals the total flow out of v.

The **value** $|f|$ is the net flow leaving s (equivalently, the net flow entering t). The **maximum flow problem** asks for a flow of maximum value.

Running example (CSES 1694 sample): vertices 1..4, $s = 1$, $t = 4$, edges 1→2 (3), 2→4 (2), 1→3 (4), 3→4 (5), 4→1 (3). The maximum flow is 6: send 2 along 1→2→4 and 4 along 1→3→4. Edge 1→2 is not saturated, but 2→4 is the bottleneck on that route.

3. Residual graph and augmenting paths

3.1 Why greedy fails

Consider s→a, s→b, a→b, a→t, b→t, all with capacity 1. If the first path found is s→a→b→t, every remaining s→t path in the original graph is blocked, and we stop with value 1. The optimum is 2 (s→a→t, s→b→t). Any correct algorithm must be able to *undo* earlier decisions.

3.2 The residual graph

Given a flow f, the **residual graph** G_f contains, for every edge (u, v):

- a **forward** edge u→v with residual capacity c(u, v) − f(u, v), if positive;
- a **backward** edge v→u with residual capacity f(u, v), if positive. Pushing along it means cancelling flow on (u, v).

An **augmenting path** is an s→t path in G_f. If its smallest residual capacity is δ > 0, we can increase forward edges and decrease the original edges of backward ones by δ, obtaining a valid flow with value |f| + δ. In the example above, after s→a→b→t the residual graph has the path s→b→a→t (using the backward edge b→a), which raises the value to 2 and cancels the flow on a→b.

3.3 Implementation: edge twins

Store each edge together with its reverse edge at consecutive indices 2k and 2k + 1 in one array. Each stores only its **remaining capacity**. The reverse of a directed edge with capacity c starts with capacity 0. Pushing δ along edge `id` does `cap[id] -= δ; cap[id ^ 1] += δ`. The flow on an original edge is the capacity of its twin (or initial capacity minus remaining capacity). For an **undirected** edge of capacity c, give both twins capacity c.

3.4 Ford-Fulkerson

Repeatedly find any augmenting path and push its bottleneck. With integer capacities each augmentation adds at least 1, so it terminates after at most |f*| iterations: O(E · |f*|) total. Two important consequences:

- **Integrality theorem**: with integer capacities, there is a maximum flow in which every f(u, v) is an integer, and Ford-Fulkerson finds one. This is what makes matching and disjoint-path reductions work.
- The running time depends on the flow value; with capacities up to 10^9 it can be far too slow if paths are chosen badly.

4. Edmonds-Karp

Edmonds-Karp is Ford-Fulkerson with the **shortest** augmenting path (fewest edges), found by BFS. Two facts (proved in CLRS 26.2):

1. The BFS distance from s to every vertex in the residual graph never decreases.
2. Each time an edge is the bottleneck of an augmenting path, the distance to its tail must increase by at least 2 before it can be a bottleneck again; so each edge is critical O(V) times.

Hence there are O(VE) augmentations, each O(E) for the BFS: **O(V E²)** total, independent of the capacities. It is short and safe for graphs with a few hundred vertices and a few thousand edges.

5. Dinic's algorithm

Dinic batches many augmentations into **phases**:

1. Run BFS from s in the residual graph to compute `level[v]` = distance from s. If t is unreachable, stop.
2. The **level graph** keeps only residual edges u→v with `level[v] = level[u] + 1`. Every shortest augmenting path lives in it.
3. Find a **blocking flow** in the level graph: augment along level-graph paths (with DFS) until every s→t path in it contains a saturated edge.

After a phase, the s–t distance strictly increases (every shortest path was blocked, and the new residual edges go "backwards" in levels), so there are at most V − 1 phases.

Current-arc optimization. During a phase, if an edge out of v is found useless (saturated, or leading to a dead end), it stays useless for the rest of the phase. Keep a pointer `it[v]` to the first edge of v not yet discarded and never revisit earlier edges. With it, a blocking flow takes O(VE), so Dinic runs in **O(V² E)**. On unit-capacity networks it is O(E · min(√E, V^{2/3})), and on unit networks such as bipartite matching graphs, **O(E √V)** — the same bound as Hopcroft-Karp.

In practice Dinic is much faster than its worst-case bound and handles V ≈ 10^4, E ≈ 10^5 comfortably in most contest problems. Make it your default.

6. Max-flow min-cut

An **s-t cut** (S, T) is a partition of V with s ∈ S and t ∈ T. Its capacity is the sum of c(u, v) over edges with u ∈ S and v ∈ T (edges from T to S do not count).

- **Weak duality**: for any flow f and any cut (S, T), $|f| \le \text{cap}(S, T)$, since every unit of flow crosses from S to T at least once (net).
- **Max-flow min-cut theorem**: the maximum flow value equals the minimum cut capacity.

Proof sketch and construction. When no augmenting path exists, let S be the set of vertices reachable from s in the residual graph, and T the rest ($t \in T$). Every original edge from S to T must be saturated (otherwise its head would be reachable), and every original edge from T to S must carry 0 flow (otherwise the backward residual edge would make its tail reachable). So the net flow across the cut equals its capacity: $|f| = \text{cap}(S, T)$, and both are optimal.

Extracting the cut in code. After Dinic finishes, the last BFS failed to reach t, so `level[v] >= 0` exactly for $v \in S$. The cut edges are the original edges (u, v) with $u \in S$ and $v \notin S$. In the running example $S = \{1, 2\}$ and the cut is $\{1\to3, 2\to4\}$ with capacity $4 + 2 = 6$.

7. Modeling

Situation	Construction
several sources/sinks	super source S with edges $S\to s_i$, super sink T with $t_i\to T$ (capacity = supply/demand or ∞)
undirected edge with capacity c	one twin pair with capacity c in both directions
capacity on a vertex v	split v into v_in → v_out with capacity c(v); in-edges go to v_in, out-edges leave v_out
maximum number of edge-disjoint s–t paths	unit capacities; answer = max flow (Menger)
vertex-disjoint paths	vertex splitting with capacity 1
minimum number of edges/vertices to delete to disconnect s, t	min cut with unit capacities (on edges / split vertices)
assignment with quotas	layered graph S → agents → tasks → T with quota capacities

The habit that prevents most modeling bugs: **decide what one unit of flow represents** (one byte, one person, one assignment, one path), then make sure every constraint in the statement corresponds to the capacity of exactly one edge on every path that unit could take.

Path reconstruction (CSES 1711 Distinct Routes): after computing the flow with unit capacities, repeatedly walk from s following original edges that carry flow, marking each used edge as consumed; each walk produces one path.

8. Bipartite matching

8.1 Via flow

Given a bipartite graph with parts L and R, add S→ℓ (capacity 1) for every $\ell \in$ L, ℓ→r (capacity 1) for every allowed pair, and r→T (capacity 1). By integrality, a maximum flow uses each vertex at most once, so the saturated L→R edges form a **maximum matching**, and its size equals the flow value. Dinic solves this in O(E $\sqrt{V}$).

8.2 Kuhn's algorithm

A **matching** is a set of edges with no shared endpoints. An **augmenting path** alternates between unmatched and matched edges and starts and ends at free vertices; flipping it increases the matching by one (this is exactly a residual augmenting path in the flow network). Berge's theorem: a matching is maximum iff no augmenting path exists.

Kuhn's algorithm tries, for each left vertex ℓ, to find an augmenting path from ℓ by DFS: for each neighbour r not yet visited in this search, if r is free or its current partner can be re-matched elsewhere (recursively), match ℓ with r. Each search is O(E), so the total is **O(V · E)**. It is about 15 lines and usually fast enough for |L|, |R| ≤ a few thousand. A common speed-up is to first match greedily, and to iterate the outer loop until no augmentation is found while resetting `used` only once per round.

8.3 König's theorem and friends

In a **bipartite** graph:

- the size of a **minimum vertex cover** (vertices touching every edge) equals the size of a **maximum matching**;
- a **maximum independent set** has size |V| − maximum matching (its complement is a vertex cover);
- if there are no isolated vertices, a **minimum edge cover** has size |V| − maximum matching.

A minimum vertex cover can be read from the min cut: with Z = vertices reachable from S in the final residual graph, the cover is (L \ Z) ∪ (R ∩ Z). None of this holds for general graphs (general matching is the blossom algorithm, World Finals 3).

9. Reference implementations

9.1 Dinic — CSES 1694 Download Speed

```cpp
#include <bits/stdc++.h>
using namespace std;

struct Dinic {
    struct Edge { int to; long long cap; };
    vector<Edge> e;
    vector<vector<int>> g;
    vector<int> level, it;
    Dinic(int n) : g(n), level(n), it(n) {}
    int addEdge(int u, int v, long long c, long long rc = 0) {
        g[u].push_back(e.size()); e.push_back({v, c});
        g[v].push_back(e.size()); e.push_back({u, rc});
        return (int)e.size() - 2;
    }
    bool bfs(int s, int t) {
        fill(level.begin(), level.end(), -1);
        queue<int> q;
        q.push(s);
        level[s] = 0;
        while (!q.empty()) {
            int v = q.front(); q.pop();
            for (int id : g[v])
                if (e[id].cap > 0 && level[e[id].to] < 0) {
                    level[e[id].to] = level[v] + 1;
                    q.push(e[id].to);
                }
        }
        return level[t] >= 0;
    }
    long long dfs(int v, int t, long long f) {
        if (v == t) return f;
        for (int& i = it[v]; i < (int)g[v].size(); i++) {
            int id = g[v][i], u = e[id].to;
            if (e[id].cap > 0 && level[u] == level[v] + 1) {
                long long d = dfs(u, t, min(f, e[id].cap));
                if (d > 0) { e[id].cap -= d; e[id ^ 1].cap += d; return d; }
            }
        }
        return 0;
    }
    long long maxflow(int s, int t) {
        long long flow = 0;
        while (bfs(s, t)) {
            fill(it.begin(), it.end(), 0);
            while (long long f = dfs(s, t, LLONG_MAX)) flow += f;
        }
        return flow;
    }
};
```

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    Dinic d(n + 1);
    for (int i = 0; i < m; i++) {
        int a, b;
        long long c;
        cin >> a >> b >> c;
        d.addEdge(a, b, c);
    }
    cout << d.maxflow(1, n) << "\n";
}
```

Sample: `4 5 / 1 2 3 / 2 4 2 / 1 3 4 / 3 4 5 / 4 1 3` → 6.

9.2 Edmonds-Karp — same problem, shorter code

```
#include <bits/stdc++.h>
using namespace std;

int n;
vector<int> to;
vector<long long> cap;
vector<vector<int>> g;

void addEdge(int u, int v, long long c) {
    g[u].push_back(to.size()); to.push_back(v); cap.push_back(c);
    g[v].push_back(to.size()); to.push_back(u); cap.push_back(0);
}

long long maxflowEK(int s, int t) {
    long long flow = 0;
    while (true) {
        vector<int> pe(n, -1);  // pe[v] = edge id used to reach v
        queue<int> q;
        q.push(s);
        pe[s] = -2;
        while (!q.empty() && pe[t] == -1) {
            int v = q.front(); q.pop();
            for (int id : g[v])
                if (cap[id] > 0 && pe[to[id]] == -1) { pe[to[id]] = id; q.push(to[id]); }
        }
        if (pe[t] == -1) return flow;
        long long d = LLONG_MAX;
        for (int v = t; v != s; v = to[pe[v] ^ 1]) d = min(d, cap[pe[v]]);
        for (int v = t; v != s; v = to[pe[v] ^ 1]) { cap[pe[v]] -= d; cap[pe[v] ^ 1] +=
d; }
        flow += d;
    }
}
```

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int m;
    cin >> n >> m;
    n++;  // vertices are 1-based
    g.assign(n, {});
    for (int i = 0; i < m; i++) {
        int a, b;
        long long c;
        cin >> a >> b >> c;
        addEdge(a, b, c);
    }
    cout << maxflowEK(1, n - 1) << "\n";
}
```

9.3 Minimum cut — CSES 1695 Police Chase

Undirected streets, remove the minimum number so that 1 and n are disconnected, and print them.

```
#include <bits/stdc++.h>
using namespace std;

struct Dinic {
    struct Edge { int to; long long cap; };
    vector<Edge> e;
    vector<vector<int>> g;
    vector<int> level, it;
    Dinic(int n) : g(n), level(n), it(n) {}
    void addEdge(int u, int v, long long c, long long rc) {
        g[u].push_back(e.size()); e.push_back({v, c});
        g[v].push_back(e.size()); e.push_back({u, rc});
    }
    bool bfs(int s, int t) {
        fill(level.begin(), level.end(), -1);
        queue<int> q;
        q.push(s);
        level[s] = 0;
        while (!q.empty()) {
            int v = q.front(); q.pop();
            for (int id : g[v])
                if (e[id].cap > 0 && level[e[id].to] < 0) {
                    level[e[id].to] = level[v] + 1;
                    q.push(e[id].to);
                }
        }
        return level[t] >= 0;
    }
    long long dfs(int v, int t, long long f) {
        if (v == t) return f;
```

```
        for (int& i = it[v]; i < (int)g[v].size(); i++) {
            int id = g[v][i], u = e[id].to;
            if (e[id].cap > 0 && level[u] == level[v] + 1) {
                long long d = dfs(u, t, min(f, e[id].cap));
                if (d > 0) { e[id].cap -= d; e[id ^ 1].cap += d; return d; }
            }
        }
        return 0;
    }
    long long maxflow(int s, int t) {
        long long flow = 0;
        while (bfs(s, t)) {
            fill(it.begin(), it.end(), 0);
            while (long long f = dfs(s, t, LLONG_MAX)) flow += f;
        }
        return flow;
    }
};

int main() {
    int n, m;
    cin >> n >> m;
    Dinic d(n + 1);
    vector<pair<int, int>> streets(m);
    for (auto& [a, b] : streets) {
        cin >> a >> b;
        d.addEdge(a, b, 1, 1);  // undirected: capacity 1 both ways
    }
    long long k = d.maxflow(1, n);
    cout << k << "\n";
    // level[] is from the final (failed) BFS: level >= 0 <=> reachable from 1
    for (auto [a, b] : streets) {
        bool ra = d.level[a] >= 0, rb = d.level[b] >= 0;
        if (ra != rb) cout << a << " " << b << "\n";
    }
}
```

Because the edge is undirected, it is in the cut whenever exactly one endpoint is on the source side. Sample: `4 5 / 1 2 / 1 3 / 2 3 / 3 4 / 1 4` → 2, edges `3 4` and `1 4`.

9.4 Kuhn's matching — CSES 1696 School Dance

```
#include <bits/stdc++.h>
using namespace std;

int n, m;
vector<vector<int>> adj;  // boys 1..n -> girls 1..m
vector<int> matchR;       // matchR[girl] = boy or 0
vector<char> used;

bool tryKuhn(int v) {
    for (int r : adj[v]) {
```

```
        if (used[r]) continue;
        used[r] = 1;
        if (matchR[r] == 0 || tryKuhn(matchR[r])) {
            matchR[r] = v;
            return true;
        }
    }
    return false;
}

int main() {
    int k;
    cin >> n >> m >> k;
    adj.assign(n + 1, {});
    for (int i = 0; i < k; i++) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
    }
    matchR.assign(m + 1, 0);
    int matching = 0;
    for (int v = 1; v <= n; v++) {
        used.assign(m + 1, 0);
        if (tryKuhn(v)) matching++;
    }
    cout << matching << "\n";
    for (int r = 1; r <= m; r++)
        if (matchR[r]) cout << matchR[r] << " " << r << "\n";
}
```

Sample: `3 2 4 / 1 1 / 1 2 / 2 1 / 3 1` → 2 pairs (e.g. `2 1` and `1 2`).

10. Common mistakes

1. **Missing or wrong reverse edges.** A directed edge's twin starts at 0; an undirected edge's twin starts at c. Twins must be at indices (2k, 2k + 1) for the `id ^ 1` trick.
2. **Overflow.** Capacities up to 10^9 with many edges: the flow value needs `long long`. Use `LLONG_MAX` (not `INT_MAX`) as the initial DFS limit when capacities are `long long`.
3. **Forgetting to reset the current-arc pointers** (`it`) at each Dinic phase, or resetting them inside the inner loop (makes it quadratic again).
4. **Reading the min cut from the wrong state.** Reachability must be computed on the final residual graph, after the last failed BFS.
5. **Counting T→S edges in the cut capacity.** Only S→T edges count.
6. **Kuhn without resetting `used`** for each new left vertex (misses augmenting paths), or resetting it inside the recursion (exponential).
7. **Modeling a vertex capacity as an edge capacity.** If a city can be used by at most c paths, split it; capping its incoming edges individually is not the same thing.

8. **Recursion depth.** Dinic's DFS depth is at most the BFS distance (≤ V), fine for V ≤ 10^5 in most judges, but be careful with very long chain graphs.

11. Practice problems

- CSES 1694 — Download Speed (https://cses.fi/problemset/task/1694)
- CSES 1695 — Police Chase (https://cses.fi/problemset/task/1695)
- CSES 1696 — School Dance (https://cses.fi/problemset/task/1696)
- CSES 1711 — Distinct Routes (https://cses.fi/problemset/task/1711)
- Kattis — maxflow (https://open.kattis.com/problems/maxflow) — also asks for the flow on each edge
- Kattis — mincut (https://open.kattis.com/problems/mincut)
- Codeforces 653D — Delivery Bears (https://codeforces.com/problemset/problem/653/D) — binary search on the answer + max flow
- Codeforces 498C — Array and Operations (https://codeforces.com/problemset/problem/498/C) — matching per prime factor
- SPOJ FASTFLOW — Fast Maximum Flow (https://www.spoj.com/problems/FASTFLOW/)
- SPOJ MATCHING — Fast Maximum Matching (https://www.spoj.com/problems/MATCHING/) — needs Dinic or Hopcroft-Karp

12. Further reading

- T. Cormen, C. Leiserson, R. Rivest, C. Stein, *Introduction to Algorithms*, chapter 26 (Maximum Flow) — proofs for Edmonds-Karp and max-flow min-cut
- cp-algorithms: "Maximum flow — Dinic's algorithm" (https://cp-algorithms.com/graph/dinic.html), "Kuhn's Algorithm for Maximum Bipartite Matching" (https://cp-algorithms.com/graph/kuhn_maximum_bipartite_matching.html)
- Antti Laaksonen, *Competitive Programmer's Handbook*, chapter 20 (flows and cuts)
- Next: Nationals 5 (min-cost flow, Hopcroft-Karp, lower bounds), World Finals 3 (Hungarian, blossom)